



## Dossier d'homologation

Channel API v2 — Intégration Channel Manager

### SOMMAIRE

---

1. Guide de démarrage rapide
  2. Référence API
  3. Workflow d'intégration
  4. Gestion des erreurs
  5. Guide sandbox
  6. FAQ technique
  7. Rapport d'homologation officiel
- 

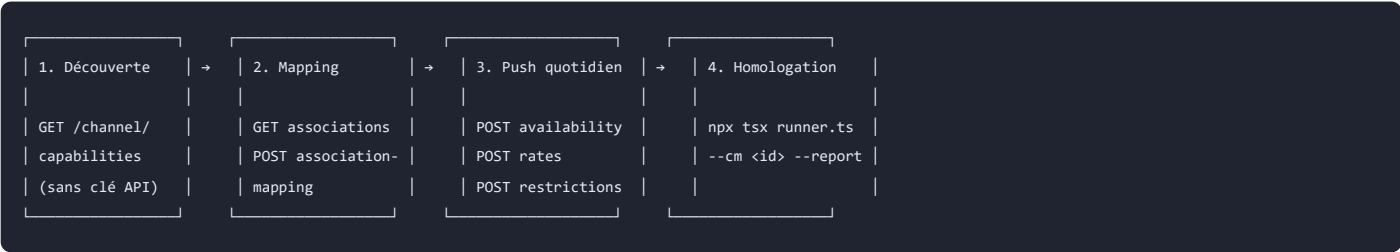
**Document généré le** 29 août 2026

**Périmètre** Fusion de docs/homologation/\*.md

**Statut homologation v2** HOMOLOGATION\_READY

Ce dossier regroupe la documentation destinée à un Channel Manager (CM) qui intègre l'API Push v2 d'otarelay : référence des endpoints, workflow d'intégration, gestion des erreurs, environnement sandbox, FAQ et rapport d'homologation officiel.

Intégration otarelay en 4 étapes



Étape 1 — Découverte ( GET /channel/capabilities )

Endpoint public, sans authentification. Il expose le contrat de l'API : champs attendus, types, contraintes, valeurs par défaut, limites ( batch\_size , rate\_limit ). C'est le point d'entrée pour générer un client ou valider votre mapping de champs avant toute intégration.

Étape 2 — Mapping ( GET / POST /channel/association-mapping )

Chaque hôtel expose des associations chambre × plan tarifaire ( rate\_plan\_room\_id ). Un CM récupère la liste via GET /channel/room-type-rate-plan-associations , puis confirme (« mappe ») chaque association qu'il gère via POST /channel/association-mapping . Seules les associations mappées acceptent des pushes.

Étape 3 — Push quotidien (disponibilités, tarifs, restrictions)

Trois endpoints indépendants, tous acceptant un objet unique ou un tableau (push par lot) : POST /channel/availability , POST /channel/rates , POST /channel/restrictions . Chaque appel est transactionnel — un enregistrement invalide dans un lot annule tout le lot (aucune écriture partielle).

Étape 4 — Homologation

Avant la mise en production, faites valider votre intégration par la suite de tests automatisée (89 tests, 8 phases). Voir homologation-report.md pour le résultat de référence et sandbox.md pour l'environnement de test.

Sommaire du dossier

| Fichier                | Contenu                                                                           |
|------------------------|-----------------------------------------------------------------------------------|
| api-reference.md       | Référence complète des 7 endpoints (méthode, auth, paramètres, réponses, erreurs) |
| workflow.md            | Détail des 3 phases d'intégration avec exemples de code                           |
| errors.md              | Tous les codes HTTP retournés par l'API, cause et solution                        |
| sandbox.md             | Environnement de test, obtention d'une clé API, données de test                   |
| faq.md                 | Questions techniques fréquentes                                                   |
| homologation-report.md | Rapport officiel des résultats de la suite d'homologation v2                      |

Convention de nommage

- CM : Channel Manager (le système partenaire qui pousse les données vers otarelay)
- rate\_plan\_room\_id : identifiant pivot d'une association chambre × plan

tarifaire — clé de voûte de tous les endpoints de push

- channel\_id : identifiant du canal, résolu automatiquement depuis votre clé API (jamais fourni dans un payload)

Support

Pour toute question technique non couverte par ce dossier, consultez d'abord faq.md , puis contactez l'équipe otarelay via votre point de contact d'intégration habituel.

Base URL sandbox : <https://api.dev.otarelay.com> (voir [sandbox.md](#) ).

Toutes les requêtes et réponses sont en JSON ( `Content-Type: application/json` ). Sauf mention contraire, chaque endpoint requiert l'en-tête :

```
Authorization: Bearer <votre_clé_api>
```

Les endpoints de push ( `/channel/availability` , `/channel/rates` , `/channel/restrictions` ) acceptent indifféremment un objet unique ou un tableau d'objets dans le corps de la requête (push par lot). Tous les éléments d'un même lot doivent porter le même `hotel_id` .

## GET /channel/capabilities

**Description** : Décrit le contrat de l'API — champs attendus par chaque endpoint de push, types, contraintes, valeurs par défaut, et limites en vigueur. Permet à un CM (ou un outil : portail partenaire, générateur de SDK, documentation automatique) de découvrir automatiquement les paramètres supportés avant de configurer son mapping et ses synchronisations.

**Authentification** : aucune (endpoint public).

**Paramètres** : aucun.

**Réponse** `200 OK` (extrait) :

```
{
  "ota_name": "otarelay",
  "version": "2.0",
  "capabilities": {
    "availability": {
      "supported": true,
      "endpoint": "POST /channel/availability",
      "batch": true,
      "fields": [
        {
          "name": "rate_plan_room_id",
          "type": "integer",
          "required": true,
          "min": 1,
          "description": "Identifiant de l'association chambre x plan tarifaire, obtenu via GET /channel/room-type-rate-plan-associations et confirmé via POST /c",
        },
        { "name": "date", "type": "string", "required": true, "description": "Date au format YYYY-MM-DD." },
        { "name": "available_rooms", "type": "integer", "required": true, "min": 0, "description": "Nombre de chambres disponibles pour cette date (0 = fermeture"
      ]
    },
    "rates": { "supported": true, "endpoint": "POST /channel/rates", "batch": true, "fields": ["..."] },
    "restrictions": { "supported": true, "endpoint": "POST /channel/restrictions", "batch": true, "fields": ["..."] }
  },
  "mapping": {
    "endpoint": "POST /channel/association-mapping",
    "delete_endpoint": "DELETE /channel/association-mapping",
    "list_endpoint": "GET /channel/room-type-rate-plan-associations"
  },
  "limits": {
    "batch_size": 366,
    "rate_limit": 1000
  }
}
```

**Codes d'erreur** : aucun cas d'erreur applicatif — l'endpoint est statique et répond toujours `200` .

## GET /channel/room-type-rate-plan-associations

**Description** : Retourne la liste des associations chambre × plan tarifaire ( `rate_plan_room_id` ) configurées pour un hôtel, avec leur statut de mapping ( `mapped: true/false` ) pour le canal ( `channel_id` ) résolu depuis votre clé API. Les associations non mappées apparaissent en premier.

**Authentification** : requise ( `Authorization: Bearer <clé>` ), la clé doit être scopée sur le `hotel_id` demandé.

**Paramètres (query)** :

Réponse 200 OK :

```
{
  "hotel_id": "HTL_TEST",
  "channel_id": "ID_REF_Siteminder",
  "associations": [
    {
      "id": 42,
      "name": "Chambre Double - Best Available Rate DBL/BAR",
      "room_type_name": "Chambre Double",
      "rate_plan_name": "Best Available Rate",
      "room_type_id": "DBL",
      "rate_plan_id": "BAR",
      "mapped": false,
      "confirmed_at": null
    }
  ],
  "summary": { "total": 9, "mapped": 0, "unmapped": 9 }
}
```

Codes d'erreur :

| Code | Cas                                                               |
|------|-------------------------------------------------------------------|
| 401  | Clé API absente, malformée ou inconnue                            |
| 403  | Hôtel inactif, ou clé API non scopée sur ce <code>hotel_id</code> |

POST /channel/association-mapping

**Description** : Confirme (« mappe ») qu'un CM gère effectivement une association chambre × plan tarifaire — précondition obligatoire avant de pouvoir pousser de la disponibilité, des tarifs ou des restrictions pour cette association. Opération **idempotente** : appeler deux fois avec le même `rate_plan_room_id` ne crée pas de doublon (re-confirmation, `confirmed_at` mis à jour).

**Authentification** : requise, clé scopée sur le `hotel_id` envoyé.

Paramètres (body) :

| Champ                          | Type    | Requis | Description                                                                                                    |
|--------------------------------|---------|--------|----------------------------------------------------------------------------------------------------------------|
| <code>hotel_id</code>          | string  | oui    | Identifiant otarelay de l'hôtel                                                                                |
| <code>rate_plan_room_id</code> | integer | oui    | Identifiant de l'association à mapper (obtenu via <code>GET /channel/room-type-rate-plan-associations</code> ) |

Requête :

```
{ "hotel_id": "HTL_TEST", "rate_plan_room_id": 42 }
```

Réponse 201 Created :

```
{
  "id": 17,
  "hotel_id": "HTL_TEST",
  "channel_id": "ID_REF_Siteminder",
  "rate_plan_room_id": 42,
  "room_type_id": "DBL",
  "rate_plan_id": "BAR",
  "confirmed_at": "2026-08-25T09:12:34.000Z"
}
```

Codes d'erreur :

| Code | Cas                                                                          |
|------|------------------------------------------------------------------------------|
| 400  | <code>rate_plan_room_id</code> absent ou non numérique (validation du corps) |
| 401  | Clé API absente, malformée ou inconnue                                       |
| 403  | Hôtel inactif, ou clé API non scopée sur ce <code>hotel_id</code>            |

DELETE /channel/association-mapping

**Description** : Retire la confirmation de mapping d'une association pour votre canal. Les pushes ultérieurs sur cette association seront rejetés ( 400 ASSOCIATION\_NOT\_MAPPED ) jusqu'à un nouveau POST .

**Authentification** : requise, clé scopée sur le hotel\_id envoyé.

**Paramètres (body)** : identiques à POST /channel/association-mapping .

```
{ "hotel_id": "HTL_TEST", "rate_plan_room_id": 42 }
```

Réponse 200 OK :

```
{ "message": "Mapping supprimé", "rate_plan_room_id": 42, "mapped": false }
```

Codes d'erreur :

| Code | Cas                                                       |
|------|-----------------------------------------------------------|
| 400  | rate_plan_room_id absent ou non numérique                 |
| 401  | Clé API absente, malformée ou inconnue                    |
| 403  | Hôtel inactif, ou clé API non scopée sur ce hotel_id      |
| 404  | Aucun mapping actif pour ce rate_plan_room_id et ce canal |

POST /channel/availability

**Description** : Pousse la disponibilité (nombre de chambres restantes) pour une ou plusieurs dates. Accepte un objet unique ou un tableau (jusqu'à 365 jours en une requête, validé en homologation). Chaque entrée doit référencer un rate\_plan\_room\_id préalablement mappé (voir POST /channel/association-mapping ).

**Authentification** : requise, clé scopée sur le hotel\_id de chaque entrée.

**Paramètres (body)** — objet ou tableau d'objets :

| Champ             | Type          | Requis | Défaut | Description                                                                  |
|-------------------|---------------|--------|--------|------------------------------------------------------------------------------|
| hotel_id          | string        | oui    | —      | Identifiant otarelay de l'hôtel (identique pour toutes les entrées d'un lot) |
| rate_plan_room_id | integer (≥ 1) | oui    | —      | Association chambre × plan tarifaire, mappée                                 |
| date              | string        | oui    | —      | Date au format YYYY-MM-DD                                                    |
| available_rooms   | integer (≥ 0) | oui    | —      | Chambres disponibles ( 0 = fermeture)                                        |

Requête (lot) :

```
[ { "hotel_id": "HTL_TEST", "rate_plan_room_id": 42, "date": "2026-09-01", "available_rooms": 5 }, { "hotel_id": "HTL_TEST", "rate_plan_room_id": 42, "date": "2026-09-02", "available_rooms": 3 } ]
```

Réponse 200 OK :

```
{ "status": "ok", "processed": 2, "failed": 0 }
```

Codes d'erreur :

| Code | Cas                                                                                                                                         |
|------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 400  | available_rooms négatif, date mal formatée, rate_plan_room_id non mappé, JSON malformé, ou une date invalide dans le lot (rollback complet) |
| 401  | Clé API absente, malformée ou inconnue                                                                                                      |
| 403  | Hôtel inactif, ou clé API non scopée sur le hotel_id du lot                                                                                 |
| 413  | Corps de requête trop volumineux                                                                                                            |
| 429  | Limite de débit dépassée (1000 req/min)                                                                                                     |

**Description :** Pousse les tarifs pour une ou plusieurs dates, avec options (devise, supplément adulte, échéance d'annulation). Même fonctionnement par lot que `/channel/availability`.

**Authentification :** requise, clé scopée sur le `hotel_id` de chaque entrée.

**Paramètres (body)** — objet ou tableau d'objets :

| Champ                              | Type               | Requis | Défaut | Description                                                                                                                                                                                      |
|------------------------------------|--------------------|--------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>hotel_id</code>              | string             | oui    | —      | Identifiant otarelay de l'hôtel                                                                                                                                                                  |
| <code>rate_plan_room_id</code>     | integer (≥ 1)      | oui    | —      | Association chambre × plan tarifaire, mappée                                                                                                                                                     |
| <code>date</code>                  | string             | oui    | —      | Date au format <code>YYYY-MM-DD</code>                                                                                                                                                           |
| <code>price</code>                 | decimal (> 0)      | oui    | —      | Prix par nuit, strictement positif                                                                                                                                                               |
| <code>currency</code>              | string (3 lettres) | non    | "EUR"  | Code devise ISO 4217                                                                                                                                                                             |
| <code>extra_adult_amount</code>    | decimal (≥ 0)      | non    | null   | Supplément par adulte additionnel                                                                                                                                                                |
| <code>cancellation_deadline</code> | datetime ISO 8601  | non    | null   | Date limite d'annulation gratuite                                                                                                                                                                |
| <code>is_refundable</code>         | boolean            | non    | —      | Accepté pour compatibilité CM mais <b>non persisté</b> : <code>is_refundable</code> reste un attribut statique du plan tarifaire côté référentiel otarelay, ignoré silencieusement si fourni ici |

**Requête :**

```
{
  "hotel_id": "HTL_TEST",
  "rate_plan_room_id": 42,
  "date": "2026-09-01",
  "price": 129.90,
  "currency": "EUR",
  "extra_adult_amount": 15,
  "cancellation_deadline": "2026-08-30T23:59:59.000Z"
}
```

**Réponse 200 OK :**

```
{ "status": "ok", "processed": 1, "failed": 0 }
```

**Codes d'erreur :**

| Code | Cas                                                                                                                                                                              |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 400  | <code>price</code> nul ou négatif, <code>date</code> mal formatée, <code>rate_plan_room_id</code> non mappé, JSON malformé, ou une ligne invalide dans le lot (rollback complet) |
| 401  | Clé API absente, malformée ou inconnue                                                                                                                                           |
| 403  | Hôtel inactif, ou clé API non scopée sur le <code>hotel_id</code> du lot                                                                                                         |
| 413  | Corps de requête trop volumineux                                                                                                                                                 |
| 429  | Limite de débit dépassée (1000 req/min)                                                                                                                                          |

POST /channel/restrictions

**Description :** Pousse les restrictions de vente (durée de séjour, fermeture de vente, blocages arrivée/départ, black-out) pour une ou plusieurs dates. Les champs `stop_sell`, `closed_to_arrival` et `closed_to_departure` sont également répercutés dans le calendrier de disponibilité (miroir interne, transparent pour le CM).

**Authentification :** requise, clé scopée sur le `hotel_id` de chaque entrée.

**Paramètres (body)** — objet ou tableau d'objets :

| Champ                          | Type          | Requis | Défaut | Description                                  |
|--------------------------------|---------------|--------|--------|----------------------------------------------|
| <code>hotel_id</code>          | string        | oui    | —      | Identifiant otarelay de l'hôtel              |
| <code>rate_plan_room_id</code> | integer (≥ 1) | oui    | —      | Association chambre × plan tarifaire, mappée |
| <code>date</code>              | string        | oui    | —      | Date au format <code>YYYY-MM-DD</code>       |

|                     |               |     |                       |                                                            |
|---------------------|---------------|-----|-----------------------|------------------------------------------------------------|
| max_stay            | integer (≥ 1) | non | — (aucune contrainte) | Durée de séjour maximale, en nuits (doit être ≥ min_stay ) |
| stop_sell           | boolean       | non | false                 | Ferme la vente pour cette date                             |
| closed_to_arrival   | boolean       | non | false                 | Interdit l'arrivée (check-in) à cette date                 |
| closed_to_departure | boolean       | non | false                 | Interdit le départ (check-out) à cette date                |
| blackout            | boolean       | non | false                 | Marque la date comme black-out                             |

Requête :

```
{
  "hotel_id": "HTL_TEST",
  "rate_plan_room_id": 42,
  "date": "2026-12-24",
  "min_stay": 2,
  "stop_sell": false,
  "closed_to_arrival": true
}
```

Réponse 200 OK :

```
{ "status": "ok", "processed": 1, "failed": 0 }
```

Codes d'erreur :

| Code | Cas                                                                                                                                                      |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 400  | min_stay = 0 , min_stay > max_stay , date mal formatée, rate_plan_room_id non mappé, JSON malformé, ou une ligne invalide dans le lot (rollback complet) |
| 401  | Clé API absente, malformée ou inconnue                                                                                                                   |
| 403  | Hôtel inactif, ou clé API non scopée sur le hotel_id du lot                                                                                              |
| 413  | Corps de requête trop volumineux                                                                                                                         |
| 429  | Limite de débit dépassée (1000 req/min)                                                                                                                  |

Pour le détail générique de chaque code HTTP (cause, message, solution), voir errors.md .

L'intégration d'un Channel Manager (CM) avec otarelay suit trois phases séquentielles. Chaque phase dépend du résultat de la précédente : la Phase 2 nécessite les identifiants découverts en Phase 1, la Phase 3 nécessite les associations confirmées en Phase 2.

Phase 1 : Discovery → Phase 2 : Mapping → Phase 3 : Push quotidien  
(une fois) (par hôtel connecté) (en continu)

Tous les exemples ci-dessous utilisent `fetch` (Node.js 18+ / navigateur) et supposent une clé API déjà obtenue (voir [sandbox.md](#)).

## Phase 1 — Discovery

**Objectif** : découvrir le contrat de l'API avant d'écrire le moindre code de mapping. Cette phase ne nécessite pas de clé API et n'est à réaliser qu'une seule fois (ou à chaque montée de version de l'intégration).

### Appel

```
const BASE_URL = 'https://api.dev.otarelay.com';

async function discoverCapabilities() {
  const res = await fetch(`${BASE_URL}/channel/capabilities`);
  if (!res.ok) {
    throw new Error(`Discovery failed: ${res.status}`);
  }
  return res.json();
}

const capabilities = await discoverCapabilities();
console.log(capabilities.limits.batch_size); // 366
console.log(capabilities.capabilities.rates.fields); // liste des champs attendus par /channel/rates
```

### Ce qu'il faut en retenir

- `capabilities.availability|rates|restrictions.fields` décrit précisément

chaque champ attendu (type, requis, défaut, contraintes min/max) — à utiliser pour générer ou valider votre mapping de champs internes.

- `capabilities.limits.batch_size` et `capabilities.limits.rate_limit`

dimensionnent vos appels par lot et votre stratégie de retry.

- Aucune authentification n'est nécessaire à cette étape.

## Phase 2 — Mapping

**Objectif** : pour chaque hôtel connecté, déterminer quelles associations chambre × plan tarifaire ( `rate_plan_room_id` ) votre CM doit gérer, puis les confirmer une par une. Cette phase se répète à chaque nouvel hôtel connecté, ou lorsque l'hôtelier modifie sa grille chambres/plans tarifaires côté otarelay.

### 2.1 — Lister les associations disponibles

```
async function listAssociations(hotelId, apiKey) {
  const res = await fetch(
    `${BASE_URL}/channel/room-type-rate-plan-associations?hotel_id=${encodeURIComponent(hotelId)}`,
    { headers: { Authorization: `Bearer ${apiKey}` } },
  );
  if (!res.ok) {
    throw new Error(`List associations failed: ${res.status}`);
  }
  return res.json();
}

const { associations, summary } = await listAssociations('HTL_TEST', apiKey);
console.log(`${summary.unmapped}/${summary.total} associations à mapper`);
```

### 2.2 — Confirmer (mapper) chaque association gérée

```

    headers: { Authorization: `Bearer ${apiKey}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ hotel_id: hotelId, rate_plan_room_id: ratePlanRoomId }),
  });
  if (!res.ok) {
    throw new Error(`Mapping failed for ${ratePlanRoomId}: ${res.status}`);
  }
  return res.json(); // 201 - { id, hotel_id, channel_id, rate_plan_room_id, room_type_id, rate_plan_id, confirmed_at }
}

// Mapper toutes les associations non mappées d'un coup
const unmapped = associations.filter((a) => !a.mapped);
for (const association of unmapped) {
  await confirmMapping('HTL_TEST', association.id, apiKey);
}

```

### 2.3 — Retirer un mapping (association non reprise par votre CM)

```

async function removeMapping(hotelId, ratePlanRoomId, apiKey) {
  const res = await fetch(`${BASE_URL}/channel/association-mapping`, {
    method: 'DELETE',
    headers: { Authorization: `Bearer ${apiKey}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ hotel_id: hotelId, rate_plan_room_id: ratePlanRoomId }),
  });
  if (!res.ok) {
    throw new Error(`Unmapping failed: ${res.status}`);
  }
  return res.json(); // 200 - { message, rate_plan_room_id, mapped: false }
}

```

#### Ce qu'il faut en retenir

- `POST /channel/association-mapping` est **idempotent** : ré-appeler sur une

association déjà mappée ne crée pas de doublon, elle est simplement reconfirmée ( `confirmed_at` mis à jour).

- Seules les associations mappées acceptent des pushes en Phase 3 — un push

sur une association non mappée est rejeté en `400 ASSOCIATION_NOT_MAPPED`.

## Phase 3 — Push quotidien

**Objectif** : synchroniser en continu disponibilités, tarifs et restrictions pour les associations mappées en Phase 2. Les trois endpoints sont indépendants (à appeler séparément) et acceptent chacun un objet unique ou un tableau (push par lot, jusqu'à `capabilities.limits.batch_size` entrées).

### 3.1 — Disponibilités

```

async function pushAvailability(entries, apiKey) {
  const res = await fetch(`${BASE_URL}/channel/availability`, {
    method: 'POST',
    headers: { Authorization: `Bearer ${apiKey}`, 'Content-Type': 'application/json' },
    body: JSON.stringify(entries), // objet unique ou tableau - tous avec le même hotel_id
  });
  if (!res.ok) {
    throw new Error(`Availability push failed: ${res.status}`);
  }
  return res.json(); // { status: 'ok', processed: N, failed: 0 }
}

await pushAvailability(
  [
    { hotel_id: 'HTL_TEST', rate_plan_room_id: 42, date: '2026-09-01', available_rooms: 5 },
    { hotel_id: 'HTL_TEST', rate_plan_room_id: 42, date: '2026-09-02', available_rooms: 3 },
  ],
  apiKey,
);

```

### 3.2 — Tarifs

```
headers: { Authorization: `Bearer ${apiKey}`, 'Content-Type': 'application/json' },
body: JSON.stringify(entries),
});
if (!res.ok) {
  throw new Error(`Rates push failed: ${res.status}`);
}
return res.json();
}

await pushRates(
  { hotel_id: 'HTL_TEST', rate_plan_room_id: 42, date: '2026-09-01', price: 129.9, currency: 'EUR' },
  apiKey,
);
```

### 3.3 — Restrictions

```
async function pushRestrictions(entries, apiKey) {
  const res = await fetch(`${BASE_URL}/channel/restrictions`, {
    method: 'POST',
    headers: { Authorization: `Bearer ${apiKey}`, 'Content-Type': 'application/json' },
    body: JSON.stringify(entries),
  });
  if (!res.ok) {
    throw new Error(`Restrictions push failed: ${res.status}`);
  }
  return res.json();
}

await pushRestrictions(
  { hotel_id: 'HTL_TEST', rate_plan_room_id: 42, date: '2026-12-24', min_stay: 2, closed_to_arrival: true },
  apiKey,
);
```

#### Bonnes pratiques de push

- **Tout-ou-rien par lot** : une entrée invalide dans un tableau annule

l'intégralité du lot (aucune écriture partielle). Validez vos payloads côté CM avant l'envoi pour éviter des lots systématiquement rejetés.

- **Idempotence par date** : chaque push est un **UPSERT** — repousser la même

date met simplement à jour la valeur existante, sans erreur ni doublon.

- **Respect du rate limit** : 1000 requêtes/minute par clé API

( `capabilities.limits.rate_limit` ). Un dépassement retourne **429** avec un en-tête **Retry-After** (secondes avant réessai).

- **Retry avec backoff exponentiel** recommandé sur toute erreur **429** ou

**5xx** transitoire.

#### Exemple de boucle de synchronisation complète

```
async function dailySync(hotelId, apiKey, availabilityEntries, rateEntries, restrictionEntries) {
  await pushAvailability(availabilityEntries, apiKey);
  await pushRates(rateEntries, apiKey);
  await pushRestrictions(restrictionEntries, apiKey);
}
```

Pour la liste exhaustive des champs et des codes d'erreur, voir `api-reference.md` et `errors.md`.

Format des réponses d'erreur

La majorité des endpoints `/channel/*` retournent une enveloppe applicative standard :

```
{ "error": "CODE_ERREUR", "message": "Description lisible", "details": { "...": "contexte additionnel" } }
```

Deux exceptions à connaître :

- **Échecs d'authentification** ( `401` / `403` ) levés par le middleware de clé

API, avant même d'atteindre le handler de route) utilisent une enveloppe légèrement différente :

```
{ "error": { "code": "INVALID_API_KEY", "message": "API key not found", "http_status": 401 } }
```

- **Erreurs de validation Zod** ( `400` ) et **JSON malformé** ( `400` ) levé par

le framework HTTP avant même le parsing applicatif) suivent le format générique de leur couche respective — voir le détail par code ci-dessous.

Tableau des codes HTTP

| Code | Cause                                                                                                                                                                                                                                                                                                                 | Message retourné                                                                                                                                                                                                                                                             | Solution recommandée                                                                                                                                                          |                                                                                                                                                        |                                                                                                                                              |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | Requête traitée avec succès ( <code>GET</code> , <code>DELETE</code> , ou push <code>POST</code> )                                                                                                                                                                                                                    | <pre>{ "status": "ok",<br/>  "processed": N, "failed": 0<br/>}</pre> (push) ou corps spécifique à l'endpoint                                                                                                                                                                 | Aucune action — traiter la réponse normalement                                                                                                                                |                                                                                                                                                        |                                                                                                                                              |
| 201  | Ressource créée ( <code>POST</code> <code>/channel/association-mapping</code> )                                                                                                                                                                                                                                       | Objet du mapping créé ( <code>id</code> , <code>hotel_id</code> , <code>channel_id</code> , <code>rate_plan_room_id</code> , ...)                                                                                                                                            | Aucune action                                                                                                                                                                 |                                                                                                                                                        |                                                                                                                                              |
| 400  | Corps de requête invalide : champ manquant/mal typé (validation Zod), JSON malformé, <code>available_rooms</code> / <code>price</code> négatif ou nul, <code>min_stay</code> > <code>max_stay</code> , date au mauvais format, ou <code>rate_plan_room_id</code> non mappé ( <code>ASSOCIATION_NOT_MAPPED</code> )    | <pre>{ "error":<br/>  "VALIDATION_ERROR",<br/>  "message": "Invalid request<br/>  input", "details": {...} }<br/>(validation) ou { "error":<br/>  "ASSOCIATION_NOT_MAPPED",<br/>  "message": "Association non<br/>  mappée", "details": {...} }<br/>(mapping manquant)</pre> | Corriger le payload selon <code>details</code> , ou confirmer d'abord le mapping via <code>POST</code> <code>/channel/association-mapping</code> avant de pousser des données |                                                                                                                                                        |                                                                                                                                              |
| 401  | En-tête <code>Authorization</code> absent ou mal formé ( <code>MISSING_AUTH_HEADER</code> ), ou clé API inconnue en base ( <code>INVALID_API_KEY</code> )                                                                                                                                                             | <pre>{ "error": { "code":<br/>  "MISSING_AUTH_HEADER" \</pre>                                                                                                                                                                                                                | <pre>"INVALID_API_KEY", "message": "...", "http_status":<br/>401 } }</pre>                                                                                                    | Vérifier la présence et le format <code>Authorization: Bearer &lt;clé&gt;</code> ; vérifier que la clé n'a pas été régénérée côté otarelay             |                                                                                                                                              |
| 403  | Clé API désactivée ( <code>KEY_DISABLED</code> ), expirée ( <code>KEY_EXPIRED</code> ), hôtel inactif ( <code>HOTEL_INACTIVE</code> ), ou clé non scopée sur le <code>hotel_id</code> demandé ( <code>INVALID_API_KEY</code> )                                                                                        | <pre>{ "error": { "code":<br/>  "KEY_DISABLED" \</pre>                                                                                                                                                                                                                       | <pre>"KEY_EXPIRED" \</pre>                                                                                                                                                    | <pre>"HOTEL_INACTIVE",<br/>... } } ou { "error":<br/>  "INVALID_API_KEY",<br/>  "message": "API<br/>  key is not scoped<br/>  to this hotel" } }</pre> | Contacte otarelay pour réactiver/renouveler la clé, ou vérifier que le <code>hotel_id</code> envoyé correspond bien à celui associé à la clé |
| 404  | <code>rate_plan_room_id</code> inexistant ou appartenant à un autre hôtel ( <code>RATE_PLAN_ROOM_NOT_FOUND</code> ), ou mapping inexistant lors d'un <code>DELETE</code> ( <code>CHANNEL_ASSOCIATION_MAPPING_NOT_FOUND</code> )                                                                                       | <pre>{ "error":<br/>  "RATE_PLAN_ROOM_NOT_FOUND"<br/>  \</pre>                                                                                                                                                                                                               | <pre>"CHANNEL_ASSOCIATION_MAPPING_NOT_FOUND",<br/>"message": "...", "details": {...} }</pre>                                                                                  | Vérifier l'identifiant via <code>GET</code> <code>/channel/room-type-rate-plan-associations</code> avant de mapper ou de démapper                      |                                                                                                                                              |
| 409  | Conflit d'unicité — n'est pas déclenché par les 7 endpoints homologués (idempotents par upsert) ; s'applique aux endpoints legacy de mapping par code ( <code>/channel/room-type-mapping</code> , <code>/channel/rate-plan-mapping</code> , dépréciés) et à la création de clé API ( <code>DUPLICATE_API_KEY</code> ) | <pre>{ "error": "DUPLICATE_MAPPING" \</pre>                                                                                                                                                                                                                                  | <pre>"DUPLICATE_API_KEY", "message": "...", "details": {...}<br/>}</pre>                                                                                                      | Ne pas recréer une ressource déjà existante ; utiliser l'endpoint de régénération le cas échéant                                                       |                                                                                                                                              |

|     |                                                                                                    | {error,message} )                                                                                                                                                         | fractionner les envois volumineux                                                                                                |
|-----|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| 429 | Limite de débit dépassée : plus de 1000 requêtes/minute pour cette clé API ( RATE_LIMIT_EXCEEDED ) | { "error": "RATE_LIMIT_EXCEEDED", "message": "Rate limit of 1000 requests/min exceeded", "details": { "retryAfterSeconds": N, "limit": 1000 } }, en-tête Retry-After: <N> | Respecter l'en-tête Retry-After avant de réessayer ; lisser les appels (batching, file d'attente) plutôt que d'émettre en rafale |

Détail par code

200 — OK

Traitement réussi. Pour les endpoints de push, le corps confirme le nombre d'entrées traitées ( processed ) ; failed reste toujours 0 sur un 200 — tout échec partiel provoque un rollback complet du lot et une réponse 400 , jamais un 200 avec failed > 0 .

201 — Created

Réservé à POST /channel/association-mapping . Un second appel avec le même rate\_plan\_room\_id retourne également 201 (upsert idempotent), pas de 409 .

400 — Bad Request

Cause la plus fréquente en intégration. Trois familles distinctes :

1. **Validation de schéma** (Zod) : champ requis absent, type incorrect, contrainte min/max violée (ex. price ≤ 0 , min\_stay = 0 , min\_stay > max\_stay , format de date invalide). Le champ details précise le ou les champs en cause.
1. **JSON malformé** : corps de requête qui n'est pas un JSON valide. Rejeté par le framework HTTP avant même d'atteindre la validation applicative.
1. **Association non mappée** ( ASSOCIATION\_NOT\_MAPPED ) : le rate\_plan\_room\_id envoyé n'a pas été confirmé via POST /channel/association-mapping pour votre canal. Sur un lot, un seul rate\_plan\_room\_id non mappé fait échouer l'ensemble de la requête (rollback).

401 — Unauthorized

Absence totale d'identité exploitable : en-tête Authorization manquant ou mal formé ( MISSING\_AUTH\_HEADER ), ou clé API syntaxiquement valide mais absente de la base otarelay ( INVALID\_API\_KEY ). À distinguer de 403 , où l'identité est reconnue mais l'accès est refusé.

403 — Forbidden

L'identité est reconnue mais l'accès est refusé :

- KEY\_DISABLED — la clé a été désactivée côté backoffice otarelay.
- KEY\_EXPIRED — la date d'expiration de la clé est dépassée.
- HOTEL\_INACTIVE — l'hôtel associé à la clé est désactivé sur la plateforme.
- Scope invalide — le hotel\_id transmis dans le corps ne correspond pas à celui associé à la clé API utilisée.

404 — Not Found

Référence inconnue dans le référentiel otarelay :

- RATE\_PLAN\_ROOM\_NOT\_FOUND — rate\_plan\_room\_id inexistant, ou existant mais rattaché à un autre hôtel que celui demandé.
- CHANNEL\_ASSOCIATION\_MAPPING\_NOT\_FOUND — tentative de DELETE sur un mapping qui n'a jamais été confirmé (ou déjà supprimé).

409 — Conflict

Non déclenché par les 7 endpoints de l'intégration standard (tous conçus en upsert idempotent). Réserve aux endpoints annexes : création de clé API sur un channel\_id déjà utilisé ( DUPLICATE\_API\_KEY ) et aux endpoints de mapping legacy dépréciés.

413 — Payload Too Large

Le corps de la requête dépasse la taille maximale acceptée par le serveur. Contrairement aux autres codes, la réponse n'est pas nécessairement au format {error, message, details} applicatif : elle peut être générée directement par le framework HTTP avant que la requête n'atteigne le code métier. Fractionnez les envois

Plus de 1000 requêtes en une minute glissante pour la même clé API. La réponse inclut un en-tête `Retry-After` (en secondes) — attendez ce délai avant de réessayer plutôt que de renvoyer immédiatement.

---

Voir `api-reference.md` pour les erreurs spécifiques à chaque endpoint et `faq.md` pour les cas d'usage fréquents.

## URL

```
https://api.dev.otarelay.com
```

Environnement de test dédié à l'intégration et à l'homologation, isolé des données de production. Tous les endpoints documentés dans [api-reference.md](#) y sont disponibles à l'identique.

## Comment obtenir une clé API

Les clés API du flux Push ne sont **pas auto-générées** par le CM : elles sont émises côté otarelay, pour un couple (hôtel, canal) donné, par un administrateur via le backoffice ou l'endpoint interne `POST /api/v1/hotels/:hotelId/api-keys`.

1. Contactez votre point de contact otarelay pour l'ouverture d'un hôtel de test sur le sandbox (ou utilisez `HTL_TEST`, voir ci-dessous).

1. otarelay génère la clé et vous la communique **une seule fois** — la clé en clair n'est jamais stockée côté serveur (seul son hash SHA-256 est conservé) et ne peut donc pas être récupérée a posteriori. Conservez-la immédiatement dans votre gestionnaire de secrets.

1. Si la clé est perdue ou compromise, demandez une régénération plutôt qu'une nouvelle création : la régénération conserve le même `channel_id` et invalide l'ancien secret.

Chaque clé est scopée sur un couple unique ( `hotel_id` , `channel_id` ) : toute requête `/channel/*` portant un `hotel_id` différent de celui associé à la clé est rejetée en `403`.

## Données de test disponibles

L'hôtel de référence utilisé par la suite d'homologation est `HTL_TEST`, préconfiguré avec :

| Référentiel                                        | Contenu                                                                                                      |
|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Chambres ( <code>room_type</code> )                | <code>DBL</code> (Double), <code>TWN</code> (Twin), <code>TPL</code> (Triple)                                |
| Plans tarifaires ( <code>rate_plan</code> )        | <code>BAR</code> (Best Available Rate), <code>NRF</code> (Non-Refundable), <code>BB</code> (Bed & Breakfast) |
| Associations pivot ( <code>rate_plan_room</code> ) | 9 combinaisons chambre × plan (3 × 3), toutes non mappées initialement pour un nouveau canal                 |

Cet hôtel et ce jeu de données sont ceux exercés par `npx tsx src/tests/homologation/runner.ts` (voir [homologation-report.md](#)) — vous pouvez vous en inspirer pour structurer vos propres scénarios de test, ou demander à otarelay un hôtel de test dédié à votre canal ( `--cm <votre_identifiant>` ).

## Comment réinitialiser le sandbox

Le sandbox ne dispose pas de réinitialisation globale automatique — chaque hôtel de test est indépendant. Deux options :

- **Réinitialiser un hôtel de test spécifique** : les mappings, disponibilités,

tarifs et restrictions sont propres à chaque association pivot ; repousser une valeur écrase simplement la précédente (upsert), sans besoin de purge préalable. Pour repartir d'un état totalement vierge, demandez à otarelay la suppression de l'hôtel de test (script interne `delete:hotel`, cascade sur toutes les données liées) puis sa recréation.

- **Retirer un mapping ponctuel** : `DELETE /channel/association-mapping`

(voir [api-reference.md](#)) pour repartir d'une association non mappée sans toucher au reste du référentiel.

## Durée de validité

Par convention, les clés API émises sur le sandbox ont une **validité de 30 jours** à compter de leur création ou dernière régénération. Passé ce délai, toute requête authentifiée avec cette clé est rejetée en `403 KEY_EXPIRED` (voir [errors.md](#)). Demandez une régénération à l'approche de l'échéance pour éviter toute interruption de vos tests d'intégration.

Une fois votre intégration validée sur ce sandbox, passez à l'étape d'homologation formelle décrite dans [homologation-report.md](#).

## 1. Pourquoi mes pushes échouent-ils systématiquement en `400 ASSOCIATION_NOT_MAPPED` ?

Le `rate_plan_room_id` utilisé n'a pas été confirmé pour votre canal. Les endpoints `/channel/availability`, `/channel/rates` et `/channel/restrictions` n'acceptent que des associations préalablement mappées via `POST /channel/association-mapping` (Phase 2 du [workflow](#)). Vérifiez le statut `mapped` via `GET /channel/room-type-rate-plan-associations` avant de pousser.

## 2. Puis-je envoyer un objet unique au lieu d'un tableau sur les endpoints de push ?

Oui. `POST /channel/availability`, `/channel/rates` et `/channel/restrictions` acceptent indifféremment un objet JSON unique ou un tableau d'objets. Le comportement (validation, transaction, réponse) est identique — un tableau d'un seul élément est équivalent à un objet unique.

## 3. Quelle est la taille maximale d'un lot ?

`capabilities.limits.batch_size` (366 dans la réponse de `GET /channel/capabilities`) documente la taille de référence, alignée sur le plus grand lot validé en homologation (une année complète, 365 jours). Aucune limite dure n'est imposée par la validation du corps de requête au-delà de la taille globale de la requête (voir [413](#)), mais restez raisonnable pour limiter la durée de traitement et le risque de rollback complet sur une seule ligne invalide.

## 4. Que se passe-t-il si une seule date est invalide dans un lot de 200 ?

Le lot entier est rejeté (`400`), aucune des 200 entrées n'est écrite. Les endpoints de push sont transactionnels « tout ou rien » — validez vos données côté CM avant l'envoi, ou fractionnez vos lots par petits groupes pour limiter l'impact d'une ligne fautive.

## 5. Puis-je repousser deux fois la même date sans erreur ?

Oui. Chaque entrée de disponibilité/tarif/restriction est un `UPSERT` sur la clé `(hotel_id, room_type, rate_plan, date)` — repousser écrase simplement la valeur précédente, sans erreur ni doublon. C'est le mécanisme normal de synchronisation quotidienne.

## 6. Pourquoi `is_refundable` que j'envoie sur `/channel/rates` n'apparaît-il jamais dans mes vérifications ?

`is_refundable` est accepté dans le payload pour compatibilité avec certains CM, mais **n'est pas persisté par date** : c'est un attribut statique du plan tarifaire dans le référentiel otarelay (configuré côté hôtelier), pas une valeur variable par jour. Le champ est simplement ignoré s'il est fourni.

## 7. Comment savoir si ma clé API a expiré ou a été désactivée ?

Les deux cas retournent `403`, mais avec des codes distincts : `KEY_EXPIRED` (date d'expiration dépassée) ou `KEY_DISABLED` (désactivée manuellement côté otarelay). Voir [errors.md](#). Dans les deux cas, contactez votre point de contact otarelay pour une régénération.

## 8. J'obtiens `403` alors que ma clé API est valide et active — pourquoi ?

Vérifiez que le `hotel_id` envoyé dans le corps de la requête correspond exactement à celui associé à votre clé API. Une clé est scopée sur un couple unique `(hotel_id, channel_id)` : toute requête portant un `hotel_id` différent est rejetée, même avec une clé par ailleurs valide.

## 9. Comment gérer la limite de 1000 requêtes/minute proprement ?

Surveillez l'en-tête `Retry-After` renvoyé sur une réponse `429` et respectez-le avant de réessayer. Pour éviter d'atteindre la limite, privilégiez le push par lot (un seul appel pour plusieurs dates) plutôt que des appels unitaires répétés, et lissez vos synchronisations dans le temps plutôt que de les émettre en rafale.

## 10. Le endpoint `GET /channel/capabilities` nécessite-t-il une clé API ?

Non. C'est le seul endpoint public de l'intégration — accessible sans authentification, pour permettre la découverte automatique du contrat de l'API avant même d'avoir obtenu une clé. Tous les autres endpoints `/channel/*` requièrent l'en-tête `Authorization: Bearer <clé>`.

Question non couverte ici ? Consultez [errors.md](#) pour le détail des codes HTTP, ou contactez l'équipe otarelay via votre point de contact d'intégration.

**Date :** 25 août 2026 **Version API testée :** v2 **Suite d'homologation :** `src/tests/homologation/runner.ts` (89 tests, 8 phases) **Environnement d'exécution :** exécution locale ( `http://localhost:3000` ), équivalent fonctionnel de l'environnement sandbox partenaire ( `https://api.dev.otarelay.com` , voir `sandbox.md` ) — même suite de tests, ciblable via `--env staging` sur ce domaine pour une validation finale avant mise en production. **Channel Managers testés :** CM Test 1, CM Test 2 (identifiants génériques d'homologation — voir `src/tests/homologation/utils.ts` )

Statut :  **HOMOLOGATION\_READY**

Résultat global

| Channel Manager | Tests exécutés | Résultat                                                                                | Tests ignorés                            | Statut                          |
|-----------------|----------------|-----------------------------------------------------------------------------------------|------------------------------------------|---------------------------------|
| CM Test 1       | 87/87          |  100 % | 2 SKIPPED ( <code>ENV_DEPENDENT</code> ) | <code>HOMOLOGATION_READY</code> |
| CM Test 2       | 87/87          |  100 % | 2 SKIPPED ( <code>ENV_DEPENDENT</code> ) | <code>HOMOLOGATION_READY</code> |

87 tests exécutés sur 87, tous réussis (100 %), pour chacun des deux Channel Managers de référence. 2 tests supplémentaires sont volontairement ignorés (voir Notes ci-dessous) — un test ignoré n'est jamais compté comme un échec.

Notes

- **Rate limit : 1001 requêtes → 429** `[ ENV_DEPENDENT ]` — Le rate limiting (1000 requêtes/minute par clé API) est validé manuellement via Redis. Le test automatique n'est pas fiable en environnement de test séquentiel : les 1001 requêtes émises l'une après l'autre dépassent la fenêtre d'une minute avant d'atteindre le seuil, ce qui invaliderait le test plutôt que de prouver l'absence de limitation. Ce comportement est identique sur les deux Channel Managers testés (Phase 1 et Phase 8).

Détail par phase (identique pour les deux Channel Managers)

| Phase                          | Tests        | Résultat                                                                                   | Portée         |
|--------------------------------|--------------|--------------------------------------------------------------------------------------------|----------------|
| Phase 1 — Authentification     | 6/7 (1 skip) |  PASSED | Phase critique |
| Phase 2 — Capabilities         | 11/11        |  PASSED | —              |
| Phase 3 — Associations         | 10/10        |  PASSED | Phase critique |
| Phase 4 — Mapping associations | 13/13        |  PASSED | Phase critique |
| Phase 5 — Disponibilités       | 12/12        |  PASSED | —              |
| Phase 6 — Tarifs               | 14/14        |  PASSED | —              |
| Phase 7 — Restrictions         | 14/14        |  PASSED | —              |
| Phase 8 — Robustesse           | 7/8 (1 skip) |  PASSED | —              |

Une phase marquée « critique » interrompt l'homologation en cas d'échec ( `stopped_early` ) — aucune des deux exécutions n'a déclenché cet arrêt.

Phase 1 — Authentification

-  Connexion avec clé valide → `200 OK`
-  Header `Authorization` manquant → `401 MISSING_AUTH_HEADER`
-  Clé inconnue → `401 INVALID_API_KEY`
-  Clé désactivée ( `is_active=false` ) → `403 KEY_DISABLED`
-  Clé expirée ( `expires_at < now()` ) → `403 KEY_EXPIRED`
-  Hôtel inactif → `403 HOTEL_INACTIVE`
-  Rate limit : 1001 requêtes → `429` — SKIPPED ( `ENV_DEPENDENT` )

Phase 2 — Capabilities

-  Endpoint accessible sans authentification → `200`
-  Réponse contient `ota_name` et `version`
-  Réponse contient `capabilities.availability` / `rates` / `restrictions`
-  Chaque capability contient `supported: true` et `batch: true`
-  Chaque champ contient `name` , `type` , `required` , `description`
-  `rate_plan_room_id` est `required: true` et sans `default` sur les 3 capabilities
-  Un paramètre optionnel avec valeur par défaut la documente (ex. `rates.currency` )

- ✔ Temps de réponse < 50 ms (endpoint statique)

### Phase 3 — Associations

- ✔ Retourne la liste des associations → 200
- ✔ Chaque association contient `id`, `name`, `room_type_name`, `rate_plan_name`, `room_type_id`, `rate_plan_id`, `mapped`, `confirmed_at`
- ✔ `mapped: false` pour toutes (aucun mapping initial)
- ✔ Champ `summary` présent { `total`, `mapped`, `unmapped` }
- ✔ `summary.unmapped = summary.total` au départ
- ✔ Tri : non-mappées en premier
- ✔ 403 si hôtel inactif
- ✔ 403 si `hotel_id` ne correspond pas à la clé API (scope)
- ✔ 401 si clé absente
- ✔ Temps de réponse < 500 ms

### Phase 4 — Mapping associations

- ✔ POST `/channel/association-mapping` — mapper une association valide → 201
- ✔ Réponse contient `id`, `hotel_id`, `channel_id`, `rate_plan_room_id`, `room_type_id`, `rate_plan_id`
- ✔ GET `associations` → `mapped: true` après POST
- ✔ `summary.mapped` +1 après chaque mapping
- ✔ Idempotent : POST deux fois → pas de doublon
- ✔ `rate_plan_room_id` inexistant → 404
- ✔ `rate_plan_room_id` d'un autre hôtel → 404
- ✔ 403 si hôtel inactif
- ✔ DELETE `/channel/association-mapping` — supprimer un mapping existant → 200
- ✔ GET `associations` → `mapped: false` après DELETE
- ✔ `summary.unmapped` +1 après DELETE
- ✔ Mapping inexistant → 404
- ✔ Workflow complet : mapper toutes les associations de l'hôtel

### Phase 5 — Disponibilités

- ✔ Push disponibilité valide → 200
- ✔ Vérification données en base après push
- ✔ Push 0 chambre disponible → 200, stocké correctement (fermeture)
- ✔ `available_rooms` négatif → 400
- ✔ Format de date invalide → 400
- ✔ `rate_plan_room_id` non mappé → 400
- ✔ 403 si hôtel inactif
- ✔ Push 30 jours en une requête → 200, `processed = 30`
- ✔ Vérification de toutes les dates en base après le lot
- ✔ UPSERT : repush même date → mise à jour
- ✔ Une date invalide dans le lot → rollback complet → 400
- ✔ Push 365 jours → 200 en moins de 5 secondes

### Phase 6 — Tarifs

- ✔ Push tarif valide → 200
- ✔ Vérification données en base
- ✔ `price = 0` → 400
- ✔ `price` négatif → 400
- ✔ `currency` manquante → défaut EUR appliqué
- ✔ `is_refundable` manquant → accepté (non persisté par date)
- ✔ `rate_plan_room_id` non mappé → 400
- ✔ 403 si hôtel inactif
- ✔ Push 30 tarifs en une requête → 200, `processed = 30`
- ✔ UPSERT : repush même date → prix mis à jour
- ✔ Rollback si une ligne invalide dans le lot → 400
- ✔ `extra_adult_amount` fourni → stocké correctement
- ✔ `cancellation_deadline` fourni → stocké correctement
- ✔ Push sans `extra_adult_amount` → null en base

### Phase 7 — Restrictions

- ✓ `min_stay = 0` → `400`
- ✓ `min_stay > max_stay` → `400`
- ✓ `stop_sell = true` → stocké correctement
- ✓ `closed_to_arrival = true` → stocké correctement
- ✓ `rate_plan_room_id` non mappé → `400`
- ✓ `403` si hôtel inactif
- ✓ Push 30 restrictions → `200`, `processed = 30`
- ✓ Rollback si une ligne invalide dans le lot
- ✓ `stop_sell = true` puis `false` → mise à jour OK
- ✓ `blackout = true` → stocké correctement
- ✓ `closed_to_arrival` sur `check_in` → propagé au miroir `calendar_availability`

## Phase 8 — Robustesse

- ✓ 50 hôtels en push simultané → pas d'erreur
- ✓ Payload JSON malformé → `400`
- ✓ Payload > 10 Mo → `413`
- ✓ Timeout simulé → géré correctement (pas de blocage indéfini)
- ✓ Retry avec backoff : 3 échecs → succès
- ⚠ Rate limit : 1001 req/min → `429` — SKIPPED ( `ENV_DEPENDENT` )
- ✓ Vérification logs d'audit générés pour chaque requête
- ✓ Vérification cache Redis invalidé après chaque push

## Conclusion

L'API Push otarelay v2 est déclarée **prête pour homologation partenaire** ( `HOMOLOGATION_READY` ) sur la base de 87 tests exécutés et réussis sur 87 (100 %), couvrant l'authentification, la découverte de capacités, le mapping d'associations, et les trois flux de push (disponibilités, tarifs, restrictions), ainsi que la robustesse (charge, payloads invalides, timeout, retry, audit, cache). Les 2 tests ignorés portent exclusivement sur la vérification automatisée du rate limiting, validée par ailleurs manuellement ; ils ne constituent pas un frein à l'homologation.

Avant mise en production d'un Channel Manager partenaire, otarelay recommande de rejouer cette même suite contre l'environnement sandbox ( <https://api.dev.otarelay.com>, `--env staging` ) avec les identifiants propres au CM concerné ( `--cm <identifiant>` ), afin de valider l'intégration réelle plutôt que le jeu de données de référence `HTL_TEST`.

## Reproduction :

```
npx tsx src/tests/homologation/runner.ts --cm cm_test1 --version v2 --report
npx tsx src/tests/homologation/runner.ts --cm cm_test2 --version v2 --report
```

Signé : **Équipe otarelay — Homologation Partenaires**